CRM Sales Opportunities Data Warehouse

Dimensional modelling and ETL with Apache Airflow, dbt and DuckDB — Project documentation by Malak Ahmed

1. Project Overview

This project builds a small analytical data warehouse for the Maven Analytics **CRM Sales Opportunities** dataset. The goal is to turn four raw CSV files into a clean star schema that can answer sales questions (revenue by product, agent performance, win rate, sales cycle length) and to automate the whole process end to end.

Requirements covered

- Dimensional modelling: conceptual, logical and physical schema.
- ETL implemented with Airflow (orchestration), dbt (transformation and testing) and DuckDB (storage engine).
- Deliverables: project code (zip) and this documentation.

2. Source Data

The dataset contains four CSV files plus a data dictionary:

File	Description	Key columns
accounts.csv	Customer companies	account, sector, year_established, revenue, employees, office_location, subsidiary_of
products.csv	Products sold	product, series, sales_price
sales_teams.csv	Sales agents and their hierarchy	sales_agent, manager, regional_office
sales_pipeline.csv	One row per sales opportunity	opportunity_id, sales_agent, product, account, deal_stage, engage_date, close_date, close_value

The business process being modelled is the **sales pipeline**: an opportunity is created for an account, assigned to an agent, linked to a product, and moves through the stages Prospecting, Engaging, Won or Lost.

3. Architecture

The pipeline follows an ELT pattern, running inside Docker through the Astronomer (Astro) CLI.

Step	Tool	What happens
1. Extract and load	Python (pandas + DuckDB) in `load_raw.py`	CSV files are loaded as-is into the `raw` schema of `crm.duckdb`
2. Staging	dbt (views)	Light cleaning and renaming of each raw table
3. Marts	dbt (views, the dbt default)	Dimensions and the fact table of the star schema
4. Quality checks	dbt tests	Uniqueness and not-null checks on the keys
5. Documentation	dbt docs	Catalog and lineage graph, generated on demand with `dbt docs generate`
Orchestration	Airflow DAG `crm_sales_etl`	Runs `load_raw`, `dbt_run` and `dbt_test` in order

Why these tools

- **DuckDB**: an embedded analytical database, no server needed, ideal for a self-contained project.
- **dbt**: SQL transformations with dependency management (`ref`, `source`), tests and auto-generated docs.
- **Airflow**: scheduling, retries, monitoring and a visual DAG.

4. Dimensional Modelling

4.1 Conceptual schema

The conceptual schema identifies the main business entities and how they relate, without attributes.

- **Sales Opportunity** (central entity) is related to:
  - **Account**: one account can have many opportunities.
  - **Product**: one product can appear in many opportunities.
  - **Sales Agent**: one agent handles many opportunities.
  - **Date**: related twice, as *Engage Date* and *Close Date*.

(Insert the conceptual schema diagram here.)

4.2 Logical schema

The logical schema is a **star schema**. The grain of the fact table is **one row per sales opportunity**.

Fact table: fact_sales_opportunities

Column	Role
opportunity_id	Primary key (degenerate dimension)
account_key	FK to dim_account
product_key	FK to dim_product
agent_key	FK to dim_sales_agent
engage_date_key	FK to dim_date (role: engage date)
close_date_key	FK to dim_date (role: close date)
deal_stage	Descriptive attribute
close_value	Measure

Dimensions

Dimension	Attributes
dim_account	account_key, account_name, sector, year_established, revenue, employees, office_location, parent_account
dim_product	product_key, product_name, series, sales_price
dim_sales_agent	agent_key, sales_agent, manager, regional_office
dim_date	date_key, full_date, year, quarter, month, month_name, day, day_of_week

Design decisions

- **Surrogate keys** (integers) are used in every dimension, so the fact table does not depend on source names.
- **Role-playing dimension**: `dim_date` is joined twice to the fact table (engage and close date) instead of creating two date tables.
- **Flattened hierarchy**: manager and regional office are stored inside `dim_sales_agent` (denormalised), which keeps the star schema simple and avoids extra joins.
- **Parent account** is kept as an attribute of `dim_account` rather than a separate snowflake table.
- **Nullable keys**: the fact table uses left joins, so no opportunity is lost. Opportunities without an account have a NULL `account_key`, and open deals have a NULL `close_date_key` and `close_value`.

(Insert the logical schema diagram here.)

4.3 Physical schema

The physical schema defines data types and constraints for DuckDB (file: `include/sql/physical_schema.sql`).

Table	Primary key	Foreign keys	Notable types
dim_account	account_key INTEGER	none	revenue DECIMAL(18,2), year_established INTEGER

Table	Primary key	Foreign keys	Notable types
dim_product	product_key INTEGER	none	sales_price DECIMAL(12,2)
dim_sales_agent	agent_key INTEGER	none	VARCHAR attributes
dim_date	date_key INTEGER (YYYYMMDD)	none	full_date DATE
fact_sales_opportunities	opportunity_id VARCHAR	account_key, product_key, agent_key, engage_date_key, close_date_key	close_value DECIMAL(18,2)

The DDL is the target design of the physical model. In the implemented pipeline dbt creates the tables with `CREATE TABLE AS` from the SQL models, so column types are inferred by DuckDB from the source data, and the keys are verified by **dbt tests** (unique, not_null) rather than by database constraints.

5. ETL Pipeline

5.1 Raw load (`load_raw.py`)

- Reads the four CSV files from `include/data/raw` with pandas.
- Registers each DataFrame in DuckDB and writes it to `raw.<table>` using `CREATE OR REPLACE TABLE`, so the load can be re-run safely (idempotent).
- Creates the `raw` schema and the `warehouse` folder if they do not exist.

5.2 dbt project (`crm_dbt`)

Staging layer: one model per source table, selecting from the matching `raw` table and applying light cleaning and renaming.

Model	Cleaning performed
stg_accounts	Fixes the sector typo `technolgy`, lowercases sector, renames `account` to `account_name` and `subsidiary_of` to `parent_account`
stg_products	Renames `product` to `product_name`
stg_sales_teams	Selects agent, manager and regional office
stg_sales_pipeline	Renames product `GTXPro` to `GTX Pro` to match the products table

Marts layer: `dim_account`, `dim_product`, `dim_sales_agent`, `dim_date` and `fact_sales_opportunities`. Surrogate keys are generated with `row_number()`. `dim_date` is built from the distinct engage and close dates found in the data. The fact table joins the staging pipeline to each dimension on its natural key and adds the surrogate keys.

(Insert the dbt lineage graph screenshot here.)

5.3 Data quality tests

Test	Applied to	Purpose
unique, not_null	account_key, product_key, agent_key, date_key (dimension keys)	Valid, non-duplicated surrogate keys
unique, not_null	opportunity_id in the fact table	One row per opportunity, no missing ids
not_null	product_key, agent_key in the fact table	Every opportunity maps to a product and an agent, which also confirms the product-name fix works

5.4 Airflow DAG (`crm_sales_etl`)

Tasks run sequentially: `load_raw >> dbt_run >> dbt_test`.

- `load_raw`: runs `load_raw.py` to load the CSV files into DuckDB.

- `dbt_run`: builds the staging and mart models.
- `dbt_test`: runs the data tests on the models.
- All tasks use a `BashOperator`; `schedule=None` means the DAG is triggered manually, and `catchup=False` avoids back-filling past runs.
- Sequential execution suits DuckDB, which allows a single writer.

(Insert the Airflow graph screenshot showing all tasks successful here.)

6. Data Quality Issues and Handling

Issue	Handling
Product name `GTXPro` in sales_pipeline vs `GTX Pro` in products	Standardised in `stg_sales_pipeline`
Sector typo `technolgy` in accounts	Corrected in `stg_accounts`
Opportunities without an account	Kept in the fact table with a NULL `account_key` (left join)
Open deals have no close date or close value	Kept as NULL

7. How to Run

1. Install Docker Desktop and the Astro CLI.
2. Place the four CSV files in `include/data/raw/`.
3. From the project root run `astro dev start`.
4. Open the Airflow UI at http://localhost:8080 (the port may differ if it was changed), enable the DAG `crm_sales_etl` and trigger it.
5. After a successful run, the warehouse is available at `include/warehouse/crm.duckdb`.
6. To explore it: `duckdb.connect('include/warehouse/crm.duckdb', read_only=True)` and query the `main` schema.

Python dependencies (`requirements.txt`): duckdb, pandas, dbt-core, dbt-duckdb.

8. Challenges and Future Improvements

Challenges

- Joining facts to dimensions on names required cleaning inconsistent values first (GTXPro, the sector typo).
- DuckDB's single-writer model requires running the tasks sequentially.
- Handling open deals with missing dates and values without losing rows.

Possible improvements

- Declare the raw tables as dbt sources (`source()`) so the lineage graph starts from the raw layer.
- Add `relationships` tests on every foreign key and `accepted_values` on `deal_stage` .
- Add an 'Unknown' member to `dim_account` so opportunities without an account never carry NULL keys.
- Add Airflow retries and a `dbt docs generate` task.
- Use `dbt_utils.generate_surrogate_key` for stable keys, and build `dim_date` from a full date spine.
- Add dbt contracts so column types (DECIMAL for amounts) match the physical DDL exactly.
- Materialize the marts as tables (they are views by default), add incremental loading, a schedule, and a dashboard on top of the star schema.

9. Appendix

- Project structure and file list: see the README in the project.
- Screenshots: Airflow DAG run, dbt lineage graph, dbt docs, conceptual and logical diagrams.